

FUNDAMENTALS · PROTOCOL ARCHITECTURE · DEEP DIVE NOTES

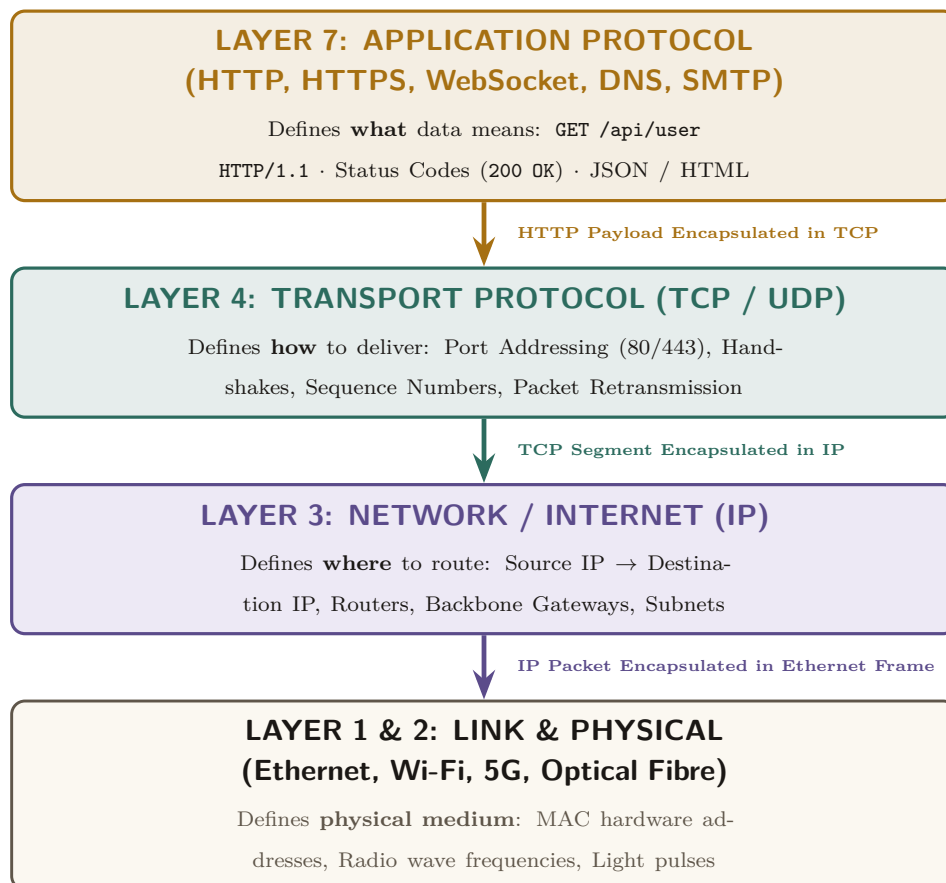
Module 3: TCP vs HTTP

The Transport Plumber vs The Application Messenger

Author & Educator Notes · Submodule 03 · Target Audience: CS / Web Engineering Students

The Classic Category Mistake: Asking “Should I use TCP or HTTP?” is like asking “Should I use a delivery truck or write a letter in English?” They do not compete with each other. HTTP is the language of the letter; TCP is the postal truck that carries it safely across the highway.

1 The Architectural Split: Layer 4 vs Layer 7



THE FUNDAMENTAL RESPONSIBILITY DIVISION**TCP (LAYER 4:)**

Focuses purely on **transmission logistics**: Opening connections, assigning sequence numbers, acknowledging packet arrival, retransmitting lost packets, and enforcing flow control. **TCP is completely blind to data meaning** — it treats HTML, video, JSON, or malware identically as raw byte streams.

HTTP (LAYER 7)

Focuses purely on **application semantics**: Understanding what the client wants (GET, POST, PUT, DELETE), status codes (200 OK, 404 Not Found), headers (Content-Type, Cookie), and payload serialization.

■ CLASSROOM METAPHOR FOR CAMERA

“Socho aapko Mumbai mein apne dost ko ek legally binding contract bhejna hai:

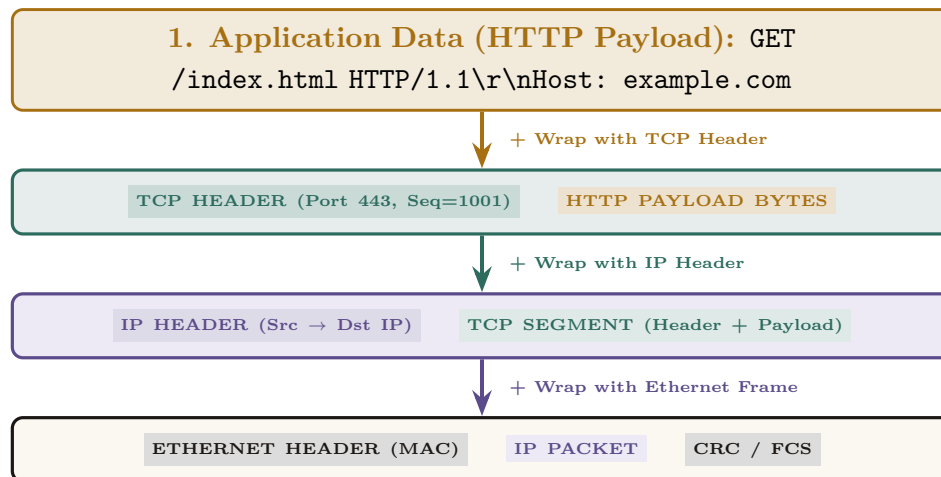
HTTP woh contract ka text hai (Title: ‘Employment Agreement’, Clause 1: ‘Salary’, Clause 2: ‘Signature’).

TCP woh Blue Dart Courier wala hai jo envelope ko barcode deta hai, sealed van mein daalta hai, OTP maangta hai, aur agar raste mein parcel barish mein bheeg gaya toh dobara dispatch karta hai.

Aap Blue Dart se contract likhne ko nahi bolte, aur contract khud chalkar Mumbai nahi jaa sakta!”

2 Data Encapsulation: How HTTP Rides Inside TCP

When your frontend application makes an API request (`fetch('/api/products')`), the browser does not send raw HTTP across the fiber optic cable directly. It wraps the data layer by layer (Encapsulation).



3 The Evolution: How HTTP Traveled Over TCP Across Generations

THE HISTORICAL PAIN OF HTTP/1.0: 1 REQUEST = 1 HANDSHAKE

In the early 1990s (HTTP/1.0), every single HTTP request opened a brand new TCP connection:

Open TCP 3-Way Handshake → Send 1 image → Close TCP Connection

If a web page had 40 images, the browser executed **40 full TCP handshakes and 40 teardowns sequentially!** The network spent more time handshaking than transferring data.

3.1 The Generational Milestones

- **HTTP/1.1 (1997 — Persistent Connections):** Introduced **Connection: keep-alive**. A single TCP connection stays open so multiple HTTP requests can be sent over the same pipe one after another.
- **HTTP/2 (2015 — Binary Framing & Multiplexing):** Broke HTTP requests into binary frames. Multiple independent HTTP requests and responses can now be interleaved simultaneously over a **single TCP socket**.
- **The Breaking Point (TCP HoL Blocking):** Even though HTTP/2 multiplexed streams, TCP underneath still treats everything as one single sequential byte stream. If one packet for Stream #1 was dropped, the OS blocked Streams #2, #3, and #4!

4 End-to-End Lifecycle: What Happens When You Fetch a Page

```
GET /api/v1/feed HTTP/1.1
Host: api.instagram.com
Accept: application/json
User-Agent: Mozilla/5.0 (iPhone)
```

5 Master Comparison Matrix: TCP vs HTTP

Dimension	TCP (Transmission Control Protocol)	HTTP (Hypertext Transfer Protocol)
OSI Layer	Layer 4 (Transport Layer)	Layer 7 (Application Layer)
Core Purpose	Moves raw bytes reliably from port to port	Expresses application intent (Resources & Actions)
Awareness	Blind to content (Raw binary stream)	Understands content (Content-Type, JSON, HTML)
State Model	State-driven (SYN_SENT, ESTABLISHED)	Stateless by default (Cookies/Tokens add state)
Addressing	Port numbers (e.g., 80, 443, 8080)	URLs and URIs (e.g., /users/42?page=1)
Primitives	SYN, ACK, FIN, RST, Window Size	GET, POST, PUT, DELETE, Status Codes
Error Codes	Reset packet (RST), Retransmit timeout	Status codes: 200 OK, 404 Not Found, 500 Server Error
Relationship	Underlying carrier that HTTP traditionally rides on	Payload language carried inside TCP segments

6 Recap: Blackboard Summary & The One-Line Spine

The One-Line Spine: Transport vs Application

CORE TAKEAWAY CHAIN FOR THE BOARD

Raw Wire: Packets drop, collide, and lack delivery guarantees →

TCP: Creates reliable, ordered, error-checked byte pipeline

TCP is blind: How do servers know what file/action client wants? →

HTTP: Provides standardized methods (GET/POST) & headers

■ ONE-SENTENCE EXAM GOLD

“Agar exam ya interview mein koi puche: ‘TCP aur HTTP mein kya difference hai?’ Toh exact ek line bolna: ‘TCP is the transport protocol that manages reliable packet delivery, while HTTP is the application protocol that defines the format of web requests and responses transmitted over that transport.’”